

```
+++ date = '2026-02-20T23:15:23-08:00' draft = false title = 'Practica3: El paradigma funcional' +++
```

Introduccion

Contexto del Problema

La gestion de tareas pendientes (To-Do List) requiere un sistema que garantice la integridad de los datos y una ejecucion libre de efectos secundarios inesperados. En lenguajes imperativos, la manipulación directa de estados globales suele generar errores dificiles de rastrear. El paradigma funcional ofrece una alternativa basada en la inmutabilidad y la evaluación de expresiones para construir sistemas mas robustos.

Objetivos

- Implementar una aplicacion de gestion de tareas utilizando el lenguaje funcional Haskell.
- Gestionar el ciclo de vida del proyecto mediante la herramienta Stack.
- Validar conceptos clave del paradigma funcional: Inmutabilidad, Funciones de Orden Superior, Tipos de Datos Algebraicos y el manejo de Entrada/Salida.

Modelo del Dominio

Arquitectura del sistema

El diseño se centra en un flujo de datos puramente funcional donde el estado de la lista de tareas no se modifica, sino que se transforma a través de funciones. Se utiliza la librería `dotenv` para la carga de configuraciones externas y `System.IO` para la interacción con el usuario.

Funciones del sistema y Responsabilidades

- **Main:** Orquestador que inicializa el entorno, carga variables desde `.env` y gestiona el bucle interactivo.
- **lookupEnv:** Funcion encargada de validar la existencia de variables de configuración, devolviendo un tipo `Maybe` para garantizar seguridad.
- **openBrowser:** Acción de IO que permite la integracion del programa con servicios externos del sistema operativo.
- **Manejo de Comandos (+, -, l, s):** Logica encargada de transformar la lista de tareas basandose en la entrada del usuario.

Evidencia de Conceptos Funcionales

Manejo de Opcionales (Maybe)

Se implemento el uso del tipo `Maybe` para gestionar variables de entorno, obligando al sistema a manejar explicitamente el caso de datos ausentes (`Nothing`) antes de continuar.

```
case website of
  Nothing -> error "You should set WEBSITE at .env file."
  Just s  -> do
```

```
result <- openBrowser s
```

Inmutabilidad y Transformacion

A diferencia de los arreglos en C, las listas en Haskell son inmutables. Cada comando de "agregar" o "eliminar" genera una nueva estructura de datos sin alterar la anterior.

Composicion y Monada IO

El uso del bloque `do` permite secuenciar acciones que tienen efectos secundarios (como leer del teclado o escribir en pantalla) manteniendo la pureza del lenguaje en el resto de la lógica.

```
main :: IO ()
main = do
    loadFile defaultConfig
```

Pruebas Manuales

```
practica03 > sesion2 > Haskell-main > examples > blog > todo > .env
1 WEBSITE=https://www.google.com
2 PORT=3000
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Escuela\PP\Portfolio\practica03\sesion2\Haskell-main\examples\blog\todo>

```
PS C:\Escuela\PP\Portfolio\practica03\sesion2\Haskell-main\examples\blog\todo> stack exec todo-exe
- <Int> - Delete the numbered entry
s <Int> - Show the numbered entry
e <Int> - Edit te numbered entry
l - List todo
r - Reverse todo
c - Clear todo
q - Quit

Test todo with Haskell. You can use +(create), -(delete), s(show), e(dit), l(list), r(everse), c(lear), q(uit) commands.
```

stack todo

The screenshot shows a VS Code editor with a project named 'Portafolio'. The Explorer sidebar on the left shows a directory structure with folders like 'practica01', 'practica02', 'practica03', 'practica04', 'data', 'i18n', 'layouts', 'public', 'resources', 'static', 'themes', and 'hugo.toml'. The main editor area shows a file named 'index.md' with the following content:

```
practica03 > session2 > Haskell-main > examples > blog > todo > .env
1 WEBSITE=https://www.google.com
2 PORT=3000
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Escuela\PP\Portafolio\practica03\session2\Haskell-main\examples\blog\todo> stack exec todo-exe
1 - List todo
r - Reverse todo
c - Clear todo
q - Quit

Test todo with Haskell. You can use +(create), -(delete), s(show), e(edit), l(list), r(everse), c(lear), q(uit) commands.
+ "Hacer reporte de la sesión 2"

Test todo with Haskell. You can use +(create), -(delete), s(show), e(edit), l(list), r(everse), c(lear), q(uit) commands.
+ "Entregar práctica 3"

Test todo with Haskell. You can use +(create), -(delete), s(show), e(edit), l(list), r(everse), c(lear), q(uit) commands.
1

"2 in total"
0: "Entregar práctica 3"
1: "Hacer reporte de la sesión 2"

Test todo with Haskell. You can use +(create), -(delete), s(show), e(edit), l(list), r(everse), c(lear), q(uit) commands.
- 0

Test todo with Haskell. You can use +(create), -(delete), s(show), e(edit), l(list), r(everse), c(lear), q(uit) commands.
1

"1 in total"
0: "Hacer reporte de la sesión 2"

Test todo with Haskell. You can use +(create), -(delete), s(show), e(edit), l(list), r(everse), c(lear), q(uit) commands.
```

Coclusion

Al terminar esta practica, lo mas interesante fue experimentar la seguridad que ofrece Haskell. En las practicas anteriores con C y Python, un error de configuración solía terminar en un "crash" o un comportamiento impredecible. Aquí, el sistema me obligó a definir correctamente el archivo .env mediante el uso de tipos como Maybe antes de siquiera permitir la ejecucion.

Entender que el codigo no "cambia" cosas, sino que "transforma" datos, cambia la perspectiva sobre como diseñar software. La separacion entre la logica pura y las acciones de IO hace que el codigo sea mucho más facil de razonar.

Referencias

- Learn You a Haskell for Great Good!. <http://learnyouahaskell.com/>
- The Haskell Tool Stack. <https://docs.haskellstack.org/en/stable/>
- Hackage: dotenv. <https://hackage.haskell.org/package/dotenv>

Mis Enlaces

- **Mi Portafolio en GitHub:** [https://github.com/kmeza1402/portafolio_PP]
- **Mi Página en Vivo:** [https://kmeza1402.github.io/portafolio_PP/practica3/]